How We Prepared Notes on the Weil Conjectures

Wei Yang

November 2026

The notes on the Weil conjectures were prepared through a collaboration between a human and AI. Without AI assistance, it would have been impossible to produce 370 pages of notes in a reasonable amount of time. We used GPT-6.1-sol to fill in mathematical details and carefully prompted Claude Opus 5.5 to improve the writing. All proof routes were determined by the human at the beginning.

1 Building the Proof Skeleton

Once we've settled on the proof route, we organize the notes around a skeleton made up of the final theorem and its key intermediate theorems. This structure serves two purposes: it shows readers the main thread of the argument, and it gives the later formalization a clear set of verification targets. With the development of LLMs, they have acquired tremendous advantages, at least in assisting with the study and retrieval of existing mathematical knowledge. This provides the conditions necessary for the following workflow.

To grasp the overall structure of a proof, we first ask an LLM to generate what might be called a rough set of lecture notes or a proof outline. This outline may be rough, but it should clarify the main components of the proof. In practice, we once attempted to refine such an outline further: starting from the key nodes in the rough notes, we asked the model to expand them recursively until every leaf node corresponded to something already available in Mathlib, thereby producing a proof that included virtually every detail. This amounted to writing a natural-language text from the perspective of formalization. The approach arose from our lack of confidence in the ability of LLMs to write Lean code: we once believed that every detail needed to be specified in advance.

In reality, however, this process proved extremely slow. It could easily become trapped in an almost endless expansion of details, with no clear criterion for when to stop, much as a formalization project can continually generate new `sorry`s without any predictable endpoint. Once a reasonable proof has been expanded to roughly ten times its original length, its details become unmanageable for a human reader. Therefore, this approach is largely unusable as a workflow.

Nevertheless, this strategy of "growing downward to Mathlib" is not entirely infeasible. The key is to skip over sufficiently large blocks of detail: that is, to select standard material that one has good reason to believe can be formalized without much difficulty and for which

detailed treatments already exist, such as certain chapters of the Stacks Project. This is sufficient to generate, for each key theorem, a plausible proof roadmap that appears to avoid circular dependencies. By fixing the main proof strategy while providing only abbreviated proofs for those parts that might otherwise require substantial external material, we obtain the skeleton of the lecture notes.

2 Writing for Mathematical Understanding

Once the skeleton is in place, the next step is to turn the proof route into text that actually conveys mathematical understanding. At this point, the LLM's writing ability becomes critical—and this ability can't be divorced from its understanding of the mathematics. After all, a skilled novelist or poet won't necessarily write mathematics well, since they may not understand the mathematical content involved (though there are rare exceptions). For this reason, before beginning the actual writing, or at least at a relatively early stage, we should have the LLM develop its own understanding of both the overall proof structure and the details of the individual arguments. Only then can the writing change the form without altering the substance.

It was precisely for this reason that we switched from GPT to Claude Opus 5.5 for the main writing process. Even without elaborate instructions, and with only basic guidance on matters such as notation and the organization of material, an LLM can produce a reasonably satisfactory text. At this stage, we can have the LLM study a large number of well-written papers and other materials to learn the writing style of particular mathematicians, thereby capturing certain rules or expectations that are difficult to articulate precisely. We can also specify an intended audience to help an LLM decide which parts deserve more detailed treatment and which can be presented more briefly.

Since human attention and energy are limited, we can additionally ask the LLM to simulate readers with the appropriate mathematical background and evaluate the text from their perspective, identifying areas in need of improvement. For those aspects of the workflow that have not yet been optimized, we believe that principles accompanied by both positive and negative examples are more effective in guiding an LLM's writing than abstract principles alone. When necessary, human authors should revise portions of the text themselves to provide concrete examples that help the LLM understand what is actually required.

Furthermore, the distinctive ideas that human authors wish to convey should be deliberately incorporated into this workflow. This is important not only for calibrating the LLM's writing, but also for preserving diversity in mathematical exposition. Indeed, there are often multiple viable paths from one statement to the next. The author's choice of a particular path, together with the decision of where to elaborate and where to omit details, is itself an expression of mathematical judgment and values. What the writing should convey is precisely these choices and the reasons behind them, rather than merely a neutral route through the proof.